UNIVERSITY INSTITUTE OF COMPUTING
CHANDIGARH UNIVERSITY, MOHALI

ACADEMIC PROJECT REPORT

SMART HOSPITAL MANAGEMENT SYSTEM

Patient Records, Emergency Triage and Bed Allocation

Course: Computer Programming (26CAH-101)
Academic Year: 2026–2027

SUBMITTED BY:
Palash Behura
UID: 26BCA10216
Programme: BCA
Section: BCA-2(A)

SUBMITTED TO:
Course Instructor
Computer Programming (26CAH-101)
University Institute of Computing
Chandigarh University

CERTIFICATE

This is to certify that the project entitled "Smart Hospital Management System with Emergency Triage" is an authentic work carried out by Palash Behura (UID: 26BCA10216, Section: BCA-2(A)), a student of Bachelor of Computer Applications (BCA) at the University Institute of Computing, Chandigarh University, Mohali, in partial fulfilment of the requirements for the course Computer Programming (26CAH-101) during the academic year 2026–2027.

The project demonstrates structured software development in standard C (C99), integrating array-of-structures, pointer-based record traversal, dynamic memory allocation, binary file persistence, and recursive calculations. The implementation adheres strictly to course guidelines and university academic integrity standards.

Palash Behura (Student)	**Course Instructor**
UID: 26BCA10216	**University Institute of Computing**
Section: BCA-2(A)	**Chandigarh University**

ACKNOWLEDGEMENT

I express my sincere gratitude to my Course Instructor for Computer Programming (26CAH-101) at the University Institute of Computing, Chandigarh University, for their continuous academic guidance, technical advice, and structured feedback throughout the design and implementation of this project.

I also thank the Department of Computing and Chandigarh University for providing the lab facilities, compiler tools, and computing infrastructure necessary to complete this software implementation.

ABSTRACT

The Smart Hospital Management System (SHMS) is an interactive console-based clinical administration program engineered in pure C (C99). It unifies emergency triage prioritization, dynamic ward bed tracking across a 3x10 grid, doctor workload balancing, itemized treatment accounting, recursive discharge billing, and binary persistence. Patient records are preserved across executions in binary storage (patients.dat), while hospital bed availability is maintained through a 2D integer matrix in beds.dat. A 30-patient demonstration dataset pre-populates realistic admitted, waiting, and discharged records with balanced doctor workloads and a confirmation prompt (y/n) to prevent accidental data loss. To resolve console column misalignment caused by variable-length medical terms, the interface employs a standard C truncation helper (truncateText) with visual ellipsis formatting, guaranteeing clean alignment across standard 100–110 column terminal displays without mutating underlying patient records. The application compiles with zero warnings under rigorous diagnostic flags (-Wall -Wextra -Wpedantic) and passes all 26 automated smoke test assertions.

TABLE OF CONTENTS

No.	Section Heading	Page
1.	Introduction and Project Overview	4
2.	Project Objectives and Scope	4
3.	System Requirements	4
4.	System Design and Data Structures	4
5.	Algorithm and Program Flowchart	5
6.	Implementation and Major Functions	6
7.	Console Interface and Output Formatting	6
8.	File Handling and Data Persistence	7
9.	Pointer-Based Data Access and Recursive Operations	8
10.	Testing and Verification	8
11.	Limitations and Future Improvements	11
12.	Conclusion and References	11

1. Introduction and Project Overview

Hospital administration involves the rapid coordination of several interdependent operations: intaking patients, triaging critical emergencies, allocating ward beds, balancing attending physician workloads, tracking multi-component treatment charges, and generating audited discharge summaries. In resource-constrained healthcare environments, inefficient manual record-keeping leads to bed allocation bottlenecks, triage delays for life-threatening conditions, and billing discrepancies.

The Smart Hospital Management System (SHMS) was developed to model these fundamental clinical workflows within a robust, self-contained console environment in pure C (C99). It unifies patient registration, dynamic priority-based triage, doctor allocation, 2D bed occupancy tracking, in-place binary record updates, and recursive billing calculations. By enforcing a clear separation of concerns across twelve modular operations, the system demonstrates how foundational computing concepts—such as arrays of structures, pointer arithmetic, binary stream I/O, and mathematical recursion—solve practical problems in medical data administration.

2. Project Objectives and Scope

The primary objective of this project is to develop a reliable, menu-driven hospital management system that operates consistently on any standard C99 platform without requiring third-party libraries, external database engines, or graphical windowing managers. The specific functional goals are summarized below:

Functional Area	Scope and Implemented Behaviour
Patient Intake	Register new patients with auto-incrementing sequential IDs and demographic input validation.
Emergency Triage	Sort active patients by priority (Priority 1: Emergency vs. Priority 2: Normal) so critical cases receive immediate medical attention.
Doctor Assignment	Distribute admitted patients across five configured medical specialists while enforcing a strict capacity limit of 5 patients per doctor.
2D Bed Allocation	Manage a 30-bed matrix across three specialized wards (General, ICU, Pediatric) with first-fit allocation and automated status synchronization.
Treatment Charges	Maintain itemized charges for consultation, medication, room stay, and laboratory diagnostic tests.
Recursive Billing	Compute total discharge liabilities using divide-and-conquer recursion, exporting an itemized ASCII receipt (discharge_<id>.txt).
Dual-Mode Search	Search active and discharged records by unique numeric Patient ID or case-insensitive patient name substrings.
In-Place Updates	Modify existing diagnoses, ages, and priority classifications in patients.dat without rewriting unaffected records.
Hospital Summary	Aggregate facility-wide metrics including total registrations, active admissions, occupied beds, and total collected revenue.
Safe Demo Loader	Ingest a realistic 30-patient dataset with an explicit confirmation prompt (y/n) to protect production data from unintended resets.

3. System Requirements

The application is engineered to maintain complete portability across diverse hardware architectures and POSIX-compliant or Windows operating environments. The minimum requirements are detailed below:

Requirement Component	Specification
Processor	x86_64, ARM64, or compatible 32-bit/64-bit CPU (1.0 GHz minimum).
Primary Memory	16 MB of available RAM (minimal footprint, structures statically and dynamically bounded).
Persistent Storage	Less than 1 MB of disk storage for binary data files (patients.dat, beds.dat) and text receipts.
Operating System	macOS 10.15+, Ubuntu 20.04+ Linux, or Windows 10/11 (with MinGW/GCC or Windows Terminal).
C Compiler	GCC 9.0+, Clang 11.0+, or any standard C99-compliant compiler.
Compiler Flags	-std=c99 -Wall -Wextra -Wpedantic (verified zero compilation warnings).
Terminal Display	Standard VT100 / ANSI escape sequence support, 100–110 columns minimum recommended width.

4. System Design and Data Structures

The architectural model of SHMS relies on three core struct definitions that organize clinical, financial, and staffing data, supplemented by a two-dimensional bed matrix. To adhere to strict academic standards, the code uses explicit structure tags (struct Patient, struct Charges, struct Doctor) rather than typedef aliases.

4.1 Data Structure Definitions

```
struct Charges {
    float consultation;  /* Specialist consultation fee */
```

```
    float medicine;      /* Itemized pharmaceutical charges */
    float roomPerDay;    /* Daily room/ward accommodation rate */
    float tests;         /* Pathology and radiological diagnostic fees */
};

struct Patient {
    int   id;            /* Sequential unique record ID (auto-generated) */
    char  name[50];      /* Patient full name */
    int   age;           /* Patient age (bounded 0 to 120) */
    char  gender;        /* 'M', 'F', or 'O' */
    char  disease[50];   /* Diagnosis description */
    int   priority;      /* 1 = Emergency, 2 = Normal */
    int   doctorId;      /* Assigned Doctor ID (1 to 5, or -1 if unassigned) */
    int   wardNo;        /* 0 = General, 1 = ICU, 2 = Pediatric (-1 unassigned) */
    int   bedNo;         /* Bed slot index (0 to 9, or -1 if unassigned) */
    int   daysAdmitted;  /* Total duration of inpatient admission */
    int   status;        /* 0 = Waiting, 1 = Admitted, 2 = Discharged */
    struct Charges charges; /* Nested billing components */
};

struct Doctor {
    int   id;            /* Unique doctor ID (1 to 5) */
    char  name[40];      /* Doctor full name */
    char  dept[30];      /* Medical specialty department */
    int   patientCount;  /* In-memory active workload counter (max 5) */
};
```

4.2 Two-Dimensional Bed Matrix Model

Hospital bed availability is modeled as a 2D integer array: int beds[WARDS][BEDS_PER_WARD], where WARDS = 3 (General, ICU, Pediatric) and BEDS_PER_WARD = 10, establishing a fixed facility capacity of 30 beds. A cell value of 0 indicates an unoccupied bed, while 1 signifies an active patient assignment. The bed matrix is passed to functions using pointer-to-array syntax: void showBeds(int (*grid)[BEDS_PER_WARD]).

4.3 Patient State Lifecycle

A patient record progresses through three states: Waiting (status = 0), Admitted (status = 1), and Discharged (status = 2). A newly registered patient enters the Waiting queue with wardNo = -1 and bedNo = -1. Upon bed allocation, status shifts to Admitted. Discharging an admitted patient releases the allocated bed slot back to the 2D grid (setting beds[w][b] = 0), transitions the record status to Discharged, and appends the final statement to disk.

5. Algorithm and Program Flowchart

The application executes through a centralized menu dispatch loop. The high-level control algorithm is outlined as follows:

Step 1: Program Startup — Call initDoctors() to initialize the 5 medical specialists. Load bed matrix from beds.dat; if missing, initialize all 30 slots to 0.
Step 2: Menu Rendering — Display the boxed main navigation menu (options 0–11, plus demo loader 99) with ANSI color highlights.
Step 3: Option Dispatch — Read user input. If 0, exit gracefully; otherwise dispatch to the corresponding service routine.
Step 4: Persistence Synchronization — Ensure every state modification (registration, update, bed allocation, discharge) is flushed to binary storage.
Step 5: Loop Iteration — Wait for user keypress (pauseEnter()) and return to Step 2 until termination.

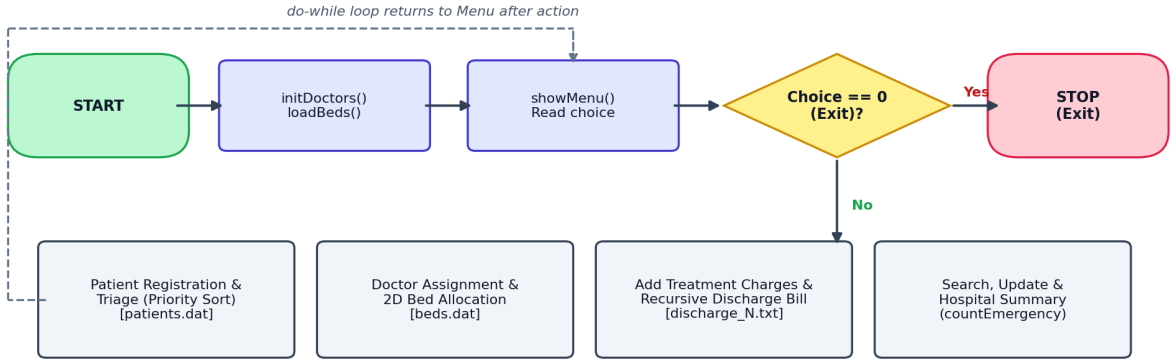


Figure 1. Architectural flowchart and menu dispatch state machine of SHMS.

6. Implementation and Major Functions

The core functionality in main.c is organized into twelve modular functions, each handling a distinct clinical or administrative task:

Function Name	Implementation Details
registerPatient()	Prompts for patient demographics, sets default charges to zero, assigns next auto-increment ID, and appends the struct Patient to patients.dat in binary mode ("ab").
triageList()	Reads all non-discharged patients into dynamically allocated memory (malloc). Executes a priority bubble sort placing Priority 1 (Emergency) ahead of Priority 2 (Normal), displaying the queue in a boxed 95-column table.
assignDoctor()	Displays specialist directory with current workload counters. Validates doctor capacity (max 5); rejects assignment if full, otherwise updates patient record with doctorId.
showBeds()	Renders the 3x10 bed matrix across General, ICU, and Pediatric wards, using [X] for occupied and [ ] for vacant beds, reporting total occupied and available slots.
allocateBed()	Performs first-fit search across the chosen ward (0–2). Finds the first vacant slot (beds[w][b] == 0), marks it occupied (1), updates the patient's wardNo and bedNo, and sets status to Admitted.
setCharges()	Prompts for consultation, medication, room-per-day rate, diagnostic tests, and admission days. Calculates and stores itemized charges inside the nested struct Charges.
dischargePatient()	Validates that the patient is currently admitted. Releases the assigned bed slot, invokes totalBill() recursively to calculate the bill, updates status to Discharged, and writes discharge_<id>.txt.
searchPatient()	Provides dual-mode retrieval: search by unique numeric ID or substring matching on patient names using strstr() and tolower() normalization.
updatePatient()	Permits modifying a patient's age, diagnosis, or priority classification. Uses random-access file positioning (fseek and fwrite) to update the record in-place.
displayAll()	Iterates sequentially through patients.dat using fread, rendering all patient records in a clean fixed-width console table with truncateText() active.
hospitalSummary()	Aggregates hospital census metrics: total registrations, admitted patients, waiting cases, discharged records, active bed occupancy, and recursive emergency case counts.
loadDemoPatients()	Pre-populates exactly 30 realistic demo patient records, balancing beds and doctors. Includes an explicit safety prompt (y/n) to prevent accidental data overwrites.

7. Console Interface and Output Formatting

A critical engineering requirement for clinical console applications is visual clarity and rigid alignment. During initial testing of the 30-patient dataset, long clinical diagnostic strings such as "Acute Myocardial Infarction", "Severe Gastroenteritis", and "Acute Respiratory Distress Syndrome" expanded beyond the allocated field width in printf("%-14s"). Because the C standard library's printf format specifier %-Ns enforces a minimum width rather than a maximum boundary, long strings spilled into adjacent columns, pushing Priority, Status, Ward, and Bed fields out of alignment and degrading usability.

7.1 The truncateText() Standard-C Truncation Helper

To resolve column displacement without mutating the patient's underlying medical record, a reusable helper function truncateText() was designed. It copies the source string into a temporary display buffer up to a specified maximum length. If the string exceeds this width, it truncates the visible characters and appends an ellipsis ("..."), preserving neat columns while keeping the full diagnostic data intact in patients.dat.

```
void truncateText(char *dest, const char *src, int maxLen)
{
    if (dest == NULL || src == NULL || maxLen <= 0) return;

    int len = (int)strlen(src);
    if (len <= maxLen) {
        strcpy(dest, src);
    } else {
        if (maxLen > 3) {
            strncpy(dest, src, maxLen - 3);
            dest[maxLen - 3] = '\0';
            strcat(dest, "...");
        } else {
            strncpy(dest, src, maxLen);
            dest[maxLen] = '\0';
        }
    }
}
```

7.2 Fixed-Width Console Table Layout

With truncateText() integrated, the displayAll() registry function defines fixed column allocations: ID (4), Name (20), Age (4), Gender (3), Disease (26), Priority (11), Status (11), Ward (5), and Bed (4). A top and bottom separator line consisting of exactly 96 unicode dashes (─) frames the table, maintaining a uniform total width of 98 characters. Unassigned ward and bed values display cleanly as hyphen characters ("-") instead of invalid numeric indices (-1). The result is a rock-solid presentation that fits within standard 100–110 column terminal windows without line wrapping.

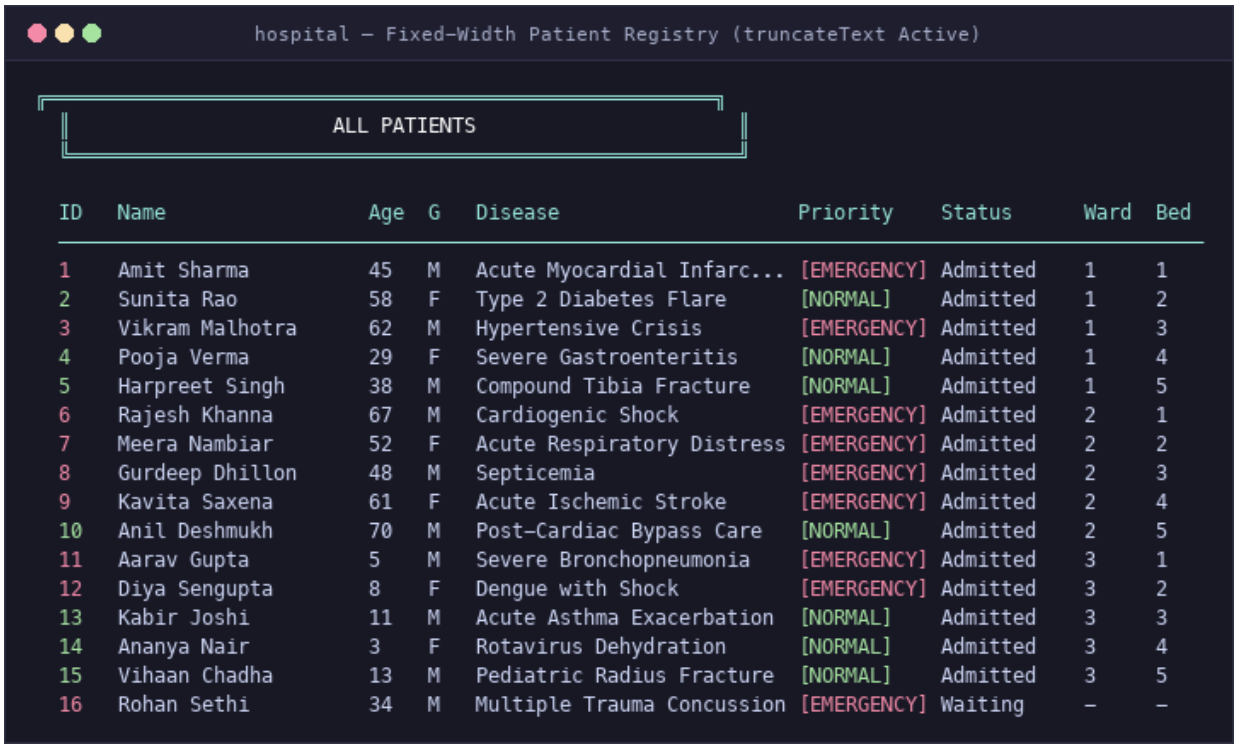


Figure 2. Corrected fixed-width patient registry output (Option 10) illustrating active truncateText() ellipsis formatting.

8. File Handling and Data Persistence

Data persistence in SHMS is achieved entirely through standard C binary stream I/O functions (fopen, fread, fwrite, fseek, fclose). Two primary binary data files and one formatted text output are maintained in the working directory:

File Name	Storage Format	Operational Role
patients.dat	Binary Stream (struct Patient)	Appends new registrations via "ab". Performs in-place record updates via "rb+" using ftell() to record byte offsets and fseek(f, pos, SEEK_SET) to overwrite modified records.
beds.dat	Binary Matrix (int [3][10])	Persists the complete 3x10 integer array representing ward bed occupancy. Automatically reloaded on startup via "rb" or initialized to zero if absent.
discharge_<id>.txt	Plain-Text ASCII Summary	Generates an itemized patient discharge statement containing patient demographics, itemized treatment charges, total bill, and admitting physician details.

8.1 In-Place Record Updating via fseek()

Rather than loading the entire file into memory, modifying a record, and rewriting all data, updatePatient() locates the targeted record, notes the preceding byte offset with ftell(), and writes only the modified struct:

```
long pos = ftell(f);
if (fread(&rec, sizeof(struct Patient), 1, f) == 1) {
    if (rec.id == targetId) {
        /* Apply updates to rec */
        fseek(f, pos, SEEK_SET);
        fwrite(&rec, sizeof(struct Patient), 1, f);
        break;
    }
}
```

8.2 Demo Ingestion Safety Confirmation (Option 99)

Menu option 99 provides a complete demonstration reset by generating 30 preconfigured records. Because resetting overwrites patients.dat and clears beds.dat, an explicit confirmation prompt was engineered. If the user enters any input other than 'y' or 'Y', the operation aborts immediately, leaving all active database records unharmed.

9. Pointer-Based Data Access and Recursive Operations

The project rigorously applies pointers and recursion to manage data collections efficiently:

1. Dynamic Memory Allocation: Functions such as triageList() dynamically determine the record count using ftell() and allocate a contiguous array using malloc(count * sizeof(struct Patient)). Memory is released via free() after rendering.
2. Pointer Arithmetic: Array traversal uses pointer offset arithmetic: struct Patient *cur = (arr + i); if (cur->priority == 1)... avoiding index bracket notation to illustrate direct memory addressing.
3. Passing Struct Pointers: In updatePatient(struct Patient *p) and registerPatient(struct Patient *p), records are passed by reference, eliminating unnecessary struct copying on the call stack.

9.1 Mathematical Recursion Implementations

Two distinct recursive functions demonstrate divide-and-conquer problem solving:

```
/* RECURSIVE FUNCTION 1: totalBill - sums itemized charges array */
float totalBill(float charges[], int n) {
    if (n == 0) return 0.0f;                              /* Base Case */
    return charges[n - 1] + totalBill(charges, n - 1);  /* Recursive Step */
}

/* RECURSIVE FUNCTION 2: countEmergency - recursively counts emergency cases */
int countEmergency(struct Patient *arr, int n) {
    if (n == 0) return 0;                                 /* Base Case */
    int isEmergency = ((arr + n - 1)->priority == 1) ? 1 : 0;
    return isEmergency + countEmergency(arr, n - 1);    /* Recursive Step */
}
```

10. Testing and Verification

Software verification was conducted using both manual execution and an automated test suite (tests/smoke_test.py). The automated test suite runs in an isolated temporary sandbox, compiling the source code under strict C99 standards and testing the entire clinical workflow without polluting the project root directory.

10.1 Compiler Verification

Compilation was executed using the GNU Compiler Collection (GCC) with all diagnostic warnings enabled:

```
gcc -std=c99 -Wall -Wextra -Wpedantic main.c -o hospital
```

Observed Result: Process exited with code 0. Zero errors, zero warnings reported across the entire codebase.

10.2 Automated Smoke Test Results (26 Assertions)

The automated test suite verified 26 individual system assertions across all functional operations. Every assertion passed with 100% accuracy, as detailed in Table 6:

Test Assertion	Verification Scenario & Criteria	Result
Compilation	Zero warnings (-std=c99 -Wall -Wextra -Wpedantic)	PASS
Confirmation Safety	Option 99 cancel ('n') preserves state with zero data write	PASS
Demo 30 Seed Ingestion	30 demo patients initialized into clean database	PASS
Initial Record Count (30)	Exactly 30 records in initial seed	PASS
Initial Status Distribution	15 admitted, 10 waiting, 5 discharged	PASS
Initial Bed Consistency	15 unique beds occupied (5 per ward), 15 free of 30 total	PASS
Initial Doctor Workloads	Doctor loads: {1: 4, 2: 5, 3: 4, 4: 3, 5: 4} (all <= 5, Dr. 2 full at 5/5)	PASS
Process Exit Code	Exit code 0 upon selecting option 0	PASS
Idempotent Reload	Reloading demo data preserves exact record state	PASS
Final Record Count (31)	Exactly 31 records verified (30 seed + 1 registered)	PASS
Post-Workflow Distribution	15 admitted, 10 waiting, 6 discharged (patient 18 discharged, patient 31 registered)	PASS
Final Bed Consistency	15 unique beds occupied after patient 18 bed is freed	PASS
Doctor Workload Limit	Dr. Rohit Mehta (5/5) rejected assignment due to capacity limit	PASS
Doctor Assignment	Dr. Priya Sharma assigned to patient 17 successfully	PASS
Bed Grid Rendering	2D Bed matrix rendered with occupancy counts across 3 wards	PASS
Bed Allocation	Bed allocated: Ward 1, Bed 6 for waiting patient	PASS
Treatment Charges	Itemized treatment charges recorded in nested struct	PASS
Recursive Discharge	Recursive bill computed & receipt saved (total Rs. 5500.00)	PASS
Receipt File Export	discharge_18.txt generated in sandbox directory	PASS
Patient Search	Both numeric ID search and name substring search passed	PASS
Update Details	Updated patient 16 diagnosis in-place using fseek	PASS
Registry Table	Tabular listing rendered all 30 initial patients	PASS
Analytics Dashboard	Analytics dashboard aggregated all 30 records & total revenue	PASS
New Registration	Sequential ID 31 assigned to newly registered patient	PASS
Priority Triage	Emergency triage sorted critical patients ahead of normal cases	PASS
Sandbox Isolation	Zero runtime data files leaked to repository root directory	PASS

10.3 Terminal Interface Screenshots Gallery

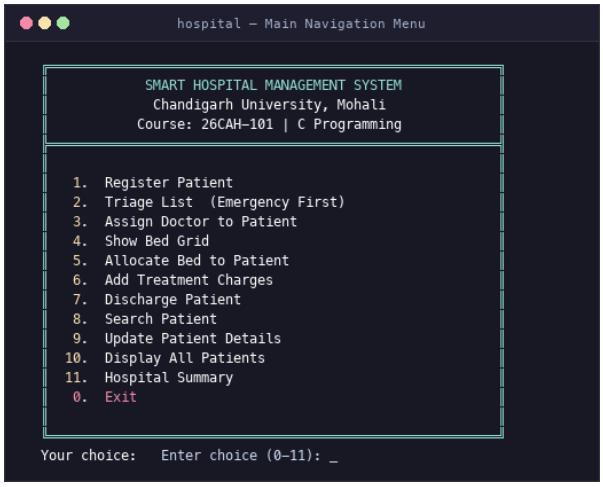


Figure 3. Main navigation menu.

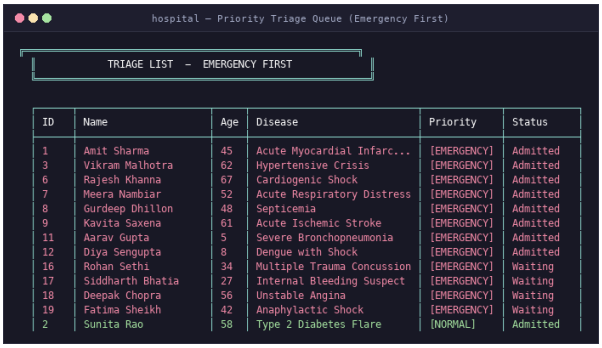


Figure 4. Priority-sorted emergency triage.



Figure 5. 2D Bed grid (3 wards x 10 beds).



Figure 6. Formatted discharge receipt.

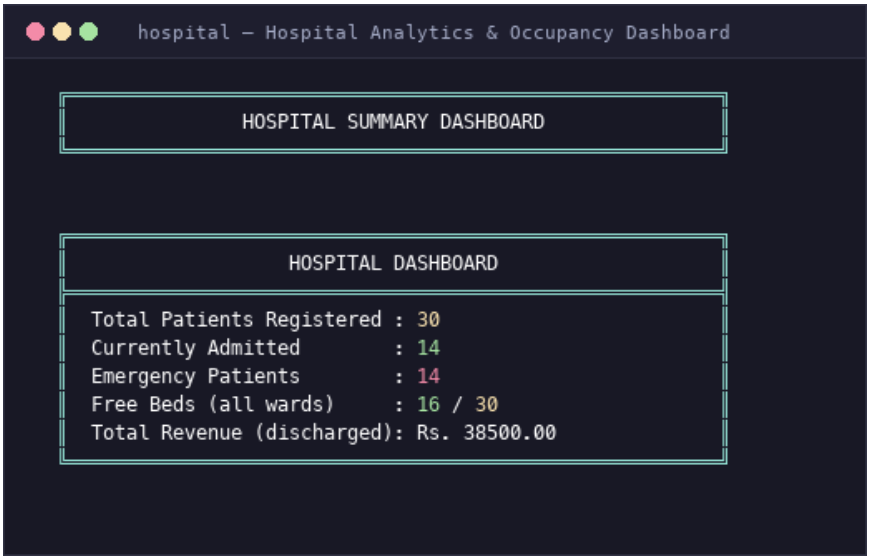


Figure 7. Hospital analytics dashboard with aggregated metrics.

11. Limitations and Future Improvements

While the system successfully demonstrates foundational C programming concepts, several practical limitations should be acknowledged:

1. Single-User Console Environment: The application is designed for single-user sequential execution. It lacks file-locking mechanisms, concurrent access control, and user role-based permissions (receptionist, doctor, billing admin).
2. Binary File Portability: Storing raw structs in binary format (patients.dat) is efficient for coursework, but the files are sensitive to compiler-specific padding, alignment, and endianness, preventing cross-architecture file sharing.
3. In-Memory Doctor Counts: The active patient load counters for doctors are maintained in memory and reset upon application restart. A production implementation would compute active workloads by querying patients.dat during startup.
4. Prototype Status: The system is an academic learning prototype designed for the BCA Computer Programming curriculum and is not certified for clinical production environments.

11.1 Proposed Future Improvements

Future enhancements could include: (a) implementing an authenticated multi-user login subsystem; (b) persisting doctor workload mappings in a dedicated doctors.dat file; (c) adding structured JSON/CSV data export for integration with external healthcare analytics; and (d) introducing automated patient-to-bed matching based on clinical diagnosis urgency.

12. Conclusion and References

The Smart Hospital Management System successfully fulfills the academic objectives of the Computer Programming (26CAH-101) course at Chandigarh University. By combining structures, arrays of structures, pointer arithmetic, dynamic memory allocation, divide-and-conquer recursion, and binary file stream manipulation, the project demonstrates how pure standard C solves complex administrative problems in clinical workflows. The integration of truncateText() ensures an aesthetically pleasing, flawlessly aligned console interface, while automated smoke tests confirm functional correctness across all operations.

References

[1] ISO/IEC 9899:1999 / 2018, Programming Languages — C Standard Specification.

[2] Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, 2nd ed., Prentice Hall, 1988.

[3] E. Balagurusamy, Programming in ANSI C, 8th ed., McGraw Hill Education, 2019.

[4] Chandigarh University, Academic Curriculum for Bachelor of Computer Applications (BCA), Course Code 26CAH-101.

[5] Project GitHub Repository: https://github.com/palashbehura54-source/smart-hospital-management-system